



LAB 6 Multiplier

Student: Alonso Emmanuel López Macías

Email's: usuario_cocyten_32_x@cinvestav.mx
alemlopezma@gmail.com

Student: Cesar Eduardo Inda Cenicerros

Email's: usuario_cocyten_32_x@cinvestav.mx
cesar.inda@iteso.mx

Full Professor: Dr. Martin Gonzalez Perez

18/04/2025

CINVESTAV, Av Del Bosque 1145, El Bajío 45017, Zapopan Jalisco.
COCYTEN, Boulevard Luis Donaldo Colosio. Ciudad Industrial, 63173 Tepic Nayarit.
Technical Report

® **CINVESTAV - COCYTEN**

Abstract: *FPGA Terasic
Board – ALTERA Cyclone IV EP4CE115F29C7N
IDE – Quartus Prime 18.1
Boolean Algebra
Digital Gates & Logical Comparators
Git & GitHub Control Version*

Keywords: *Multiplier, Partial Products, Unsigned Integers, Signed Integers, Display, Carry.*

Table of Contents

Table of Contents.....	ii
Table of Figures	iii
1.- INTRODUCTION	0
1.1.- Requirements.....	0
2.- FUNCTIONAL DESCRIPTION	1
2.1.- Boolean Algebra Equations.....	4
2.2.- Results	5
2.3.- GitHub Repository	6
3.- CHALLENGUES/ISSUES.....	6
4.- CONCLUSIONS.....	7

Table of Figures

<i>Figure: 1 Requeriments Lab6</i>	0
<i>Figure: 2 8Bit_Adder</i>	1
<i>Figure: 3 ParcialSum</i>	1
<i>Figure: 4 Multiplier</i>	2
<i>Figure: 5 Shifter</i>	2
<i>Figure: 6 PinPlanner</i>	3
<i>Figure: 7 RTL_Viewver</i>	3
<i>Figure: 8 Math Expression</i>	4
<i>Figure: 9 Table Test</i>	5
<i>Figure: 10 Test1 & Test2</i>	5
<i>Figure: 11 Test3 & Test4</i>	5
<i>Figure: 12 GitHub Repository</i>	6

1.- INTRODUCTION

In this lab, we were asked to implement a 4-bit by 4-bit multiplier, obtaining an 8-bit product as the output. The importance of the multiplier lies precisely in deriving the logic of gates to obtain these partial sums or operations and generate the final output through the total sum of those partial results. For this, it is essential to clearly understand the functionality of binary additional elements, since together with the rest of the gate logic, they form the logic of an n-bit multiplier. This involves considering previous practices using these BSF elements, applying them in this lab, and integrating them into the final design.

The hardware component of an FPGA that performs these fundamental operations is used in digital signal processing, cryptography, and even the development of embedded systems. It enables the acceleration of computations where performance bottlenecks often occur in digital systems. This helps achieve real-time efficiency in many systems, optimizing chip resources when working with large operands.

1.1.- Requirements

Design a multiplier circuit for two numbers (A and B) using the partial product sum method with the following characteristics:

- Consider the multiplier inputs as 4-bit unsigned integers.
- Consider the multiplier output as an 8-bit unsigned integer.
- Compute the partial products.
- Perform partial Sums.
- Assign the result to the output
- Implement the circuit on the FPGA.
- Verify the operations with the table test datas.

Use switches **SW[14] to SW[17]** for input data A.
Use switches **SW[0] to SW[3]** for input data B.
Display input A using LEDs **LEDR[14] to LEDR[17]**.
Display input B using LEDs **LEDR[0] to LEDR[4]**.
Display the output data using LEDs **LEDG[0] to LEDG[7]**.

Figure: 1 Requeriments Lab6

2.- FUNCTIONAL DESCRIPTION

To begin this project, the libraries or elements created in previous labs using BSF components were reused. In this lab, the previously developed 8-bit adder was used to integrate most of the required functionalities and to allow the addition of partial products by adding the remaining logic to the circuit.

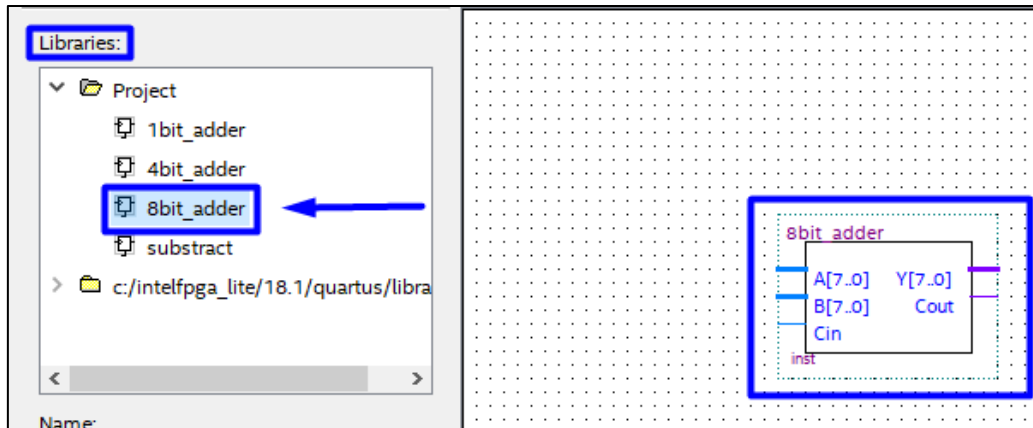


Figure: 2 8Bit_Adder

Based on this, the logic considered to obtain the "first partial sum" = "sum1" involves multiplying $A[3..0] \times B[0] = \text{"Partial0"}$ and $A[3..0] \times B[1] = \text{"Partial1"}$, resulting in $\text{"Partial0"} + \text{"Partial1"} = \text{"sum1"}$.

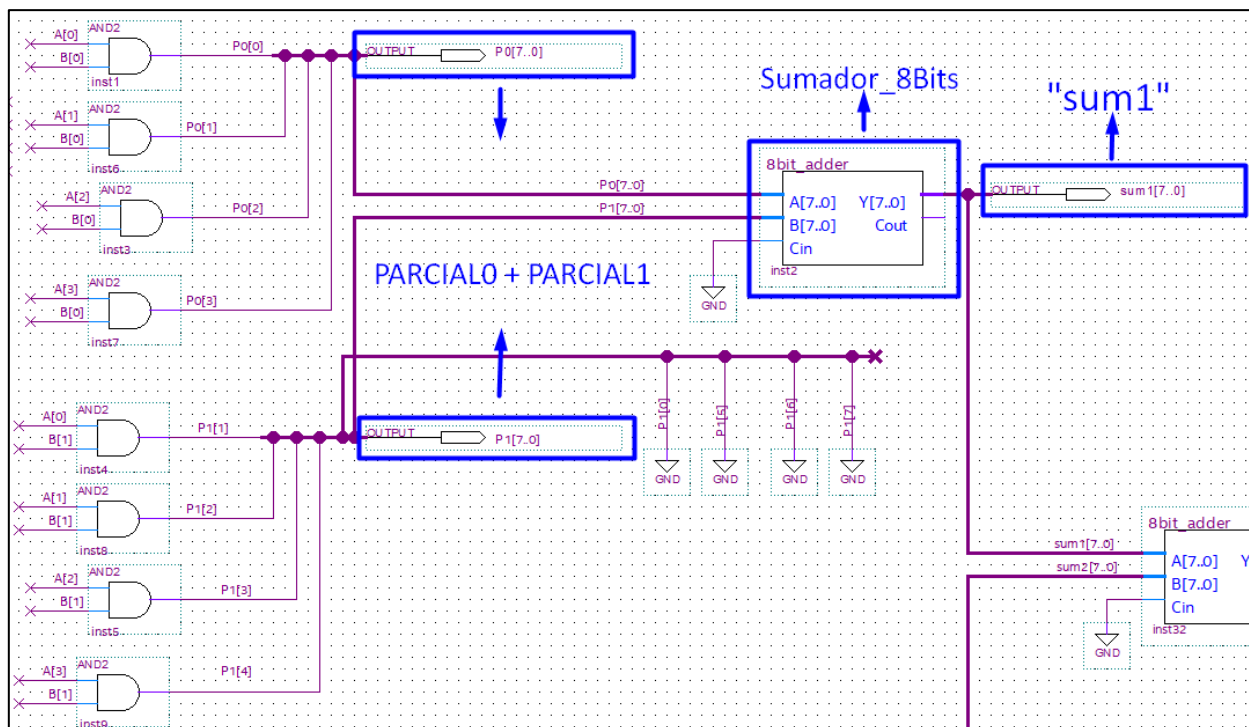


Figure: 3 ParcialSum

Thus, the final circuit was structured as follows, considering that these elements were extended throughout the schematic, and together with the rest of the gate logic, it was possible to implement this multiplier circuit.

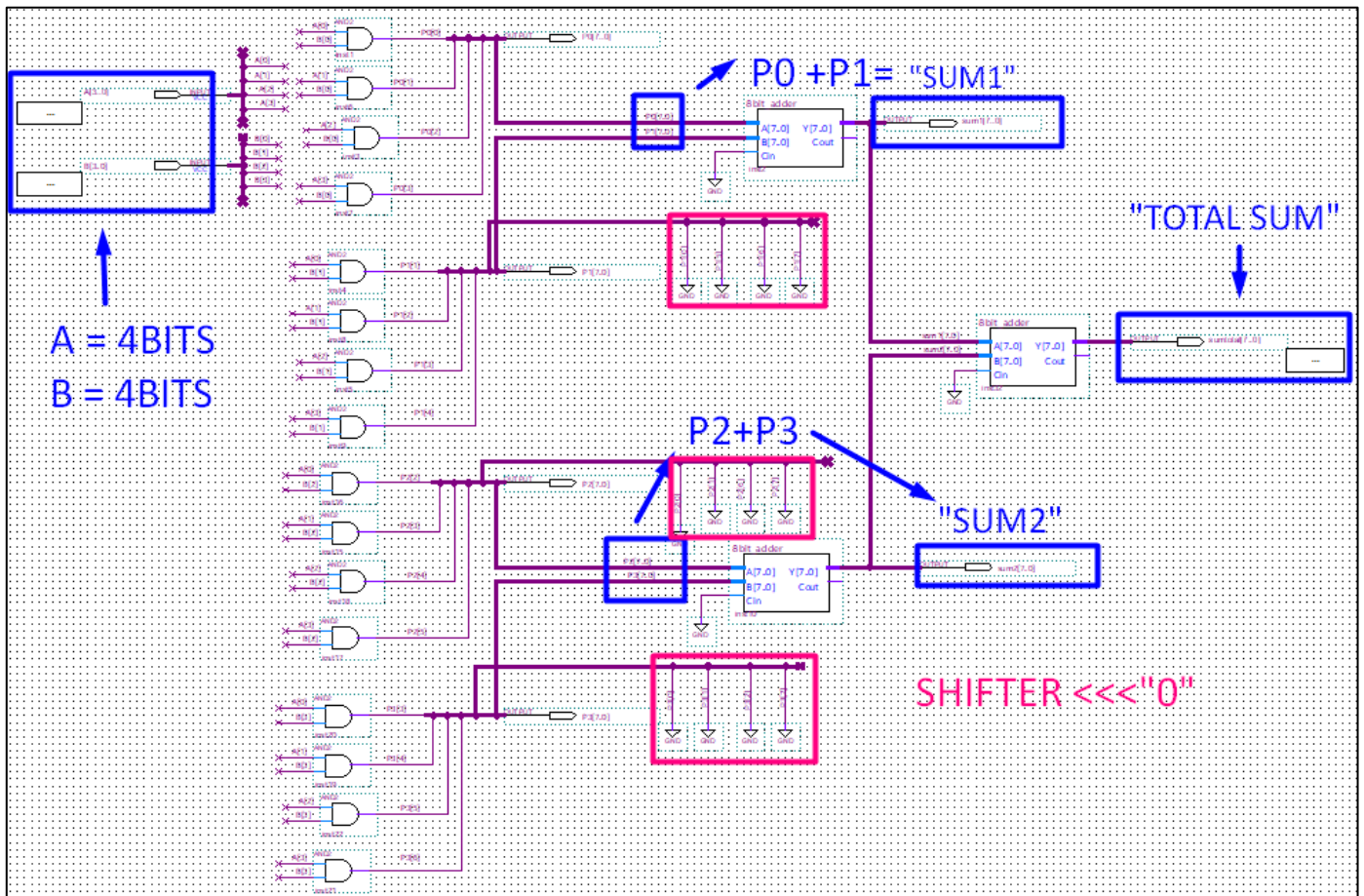


Figure: 4 Multiplier

For this implementation, a constant "0" was also connected to GND so that the Cin (Carry-in) value would be ignored. GND was also applied as "0" to the shifter elements during the operation, for the bits required at the moment of generating the project.

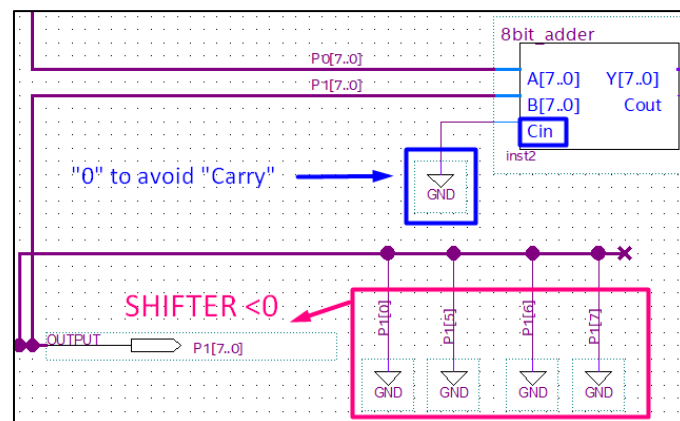


Figure: 5 Shifter

The pin assignment in the “Pin Planner” was considered only for the values that are input in the FPGA because the others are represented in the internal operations, the distrubition was as follows:

Node Name	Direction	Location
in A[3]	Input	PIN_Y23
in A[2]	Input	PIN_Y24
in A[1]	Input	PIN_AA22
in A[0]	Input	PIN_AA23
in B[3]	Input	PIN_AD27
in B[2]	Input	PIN_AC27
in B[1]	Input	PIN_AC28
in B[0]	Input	PIN_AB28
out sumtotal[7]	Output	PIN_G21 7
out sumtotal[6]	Output	PIN_G22 7
out sumtotal[5]	Output	PIN_G20 7
out sumtotal[4]	Output	PIN_H21 7
out sumtotal[3]	Output	PIN_E24 7
out sumtotal[2]	Output	PIN_E25 7
out sumtotal[1]	Output	PIN_E22 7
out sumtotal[0]	Output	PIN_E21 7

Figure: 6 PinPlanner

RTL VIEWER

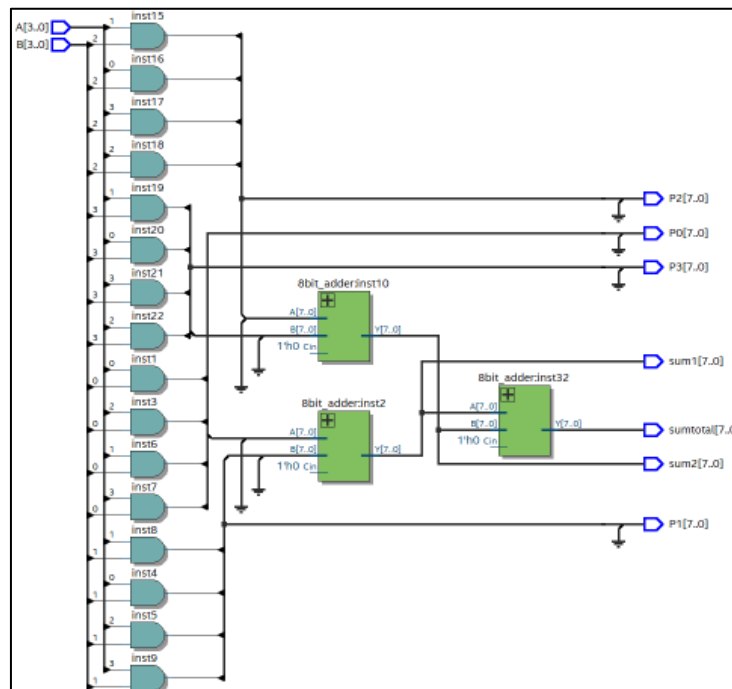


Figure: 7 RTL_Viewver

2.1.- Boolean Algebra Equations

For define the expressions was necessary first understand the functionality of full addder of 8bits, because this represents a key component on the multiplier implementation.

For this the general expression for each one bit from one variable to another.

$A = 4 \text{ BITS } (0000 - 1111).$

$B = 4 \text{ BITS } (0000 - 1111).$

That represents the following:

$(A[3..0] \times B[0]) \& (A[3..0] \times B[1]) = \text{Partial0} + \text{Partial1} = \text{Sum1}.$

$(A[3..0] \times B[2]) \& (A[3..0] \times B[3]) = \text{Partial2} + \text{Partial3} = \text{Sum2}.$

$\text{Sum1} + \text{Sum2} = \text{Sum Total}.$

The extra considerations for each were the shifters for everyone situation and "0" constant value for "Cin" and these parameters.

- $B[1] \text{ -- } <0 \text{ Shifter}[1]$
- $B[2] \text{ -- } <<0 \text{ Shifter}[2]$
- $B[3] \text{ -- } <<<0 \text{ Shifter}[3]$

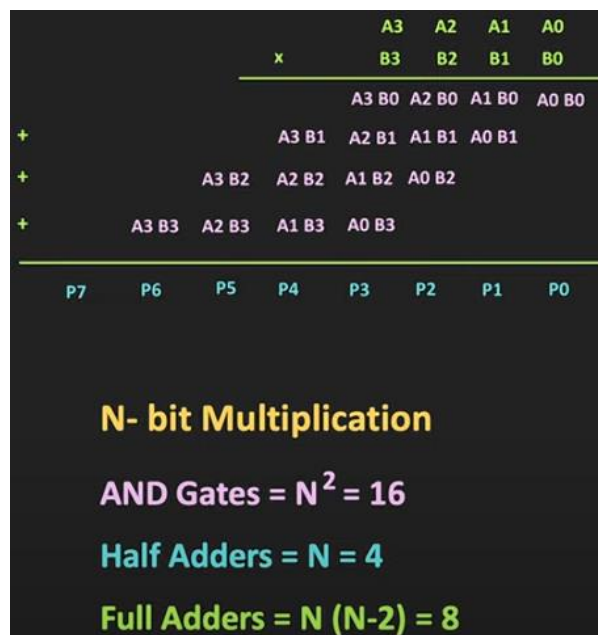


Figure: 8 Math Expression

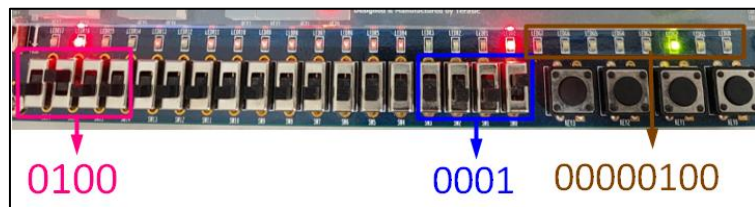
2.2.- Results

For the testing phase, the following values were considered in the tables during debugging to verify the correct functionality of the implementation.

A	B	A*B (expected)
4 (4'b0100)	1 (4'b0001)	4 (8'b00000100)
9 (4'b1001)	3 (4'b0011)	27 (8'b00011011)
13 (4'b1101)	13 (4'b1101)	169 (8'b10101001)

Figure: 9 Table Test

A=4 x B=1; Result =4



A=9 x B=3; Result =27

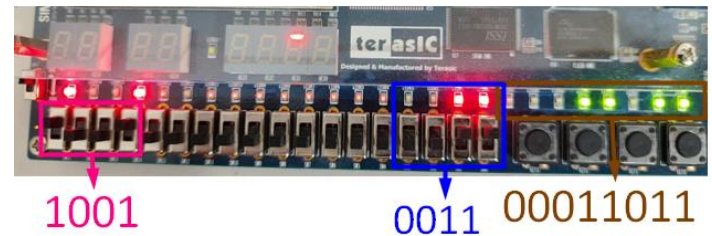
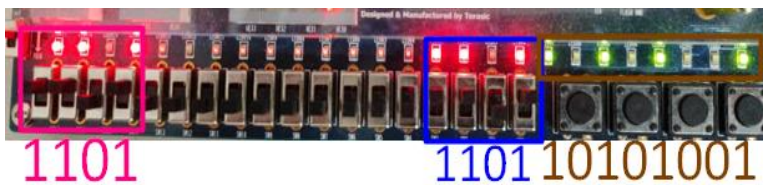


Figure: 10 Test1 & Test2

A=13 x B=13; Result =169



A=15 x B=7; Result =105



Figure: 11 Test3 & Test4

2.3.- GitHub Repository

A branch was created in the course repository corresponding to Lab6_Multiplier.

https://github.com/CeicGitHub/Fundamentals_Of_Digital_Design_FPGA/tree/QuartusLab6_Multiplier

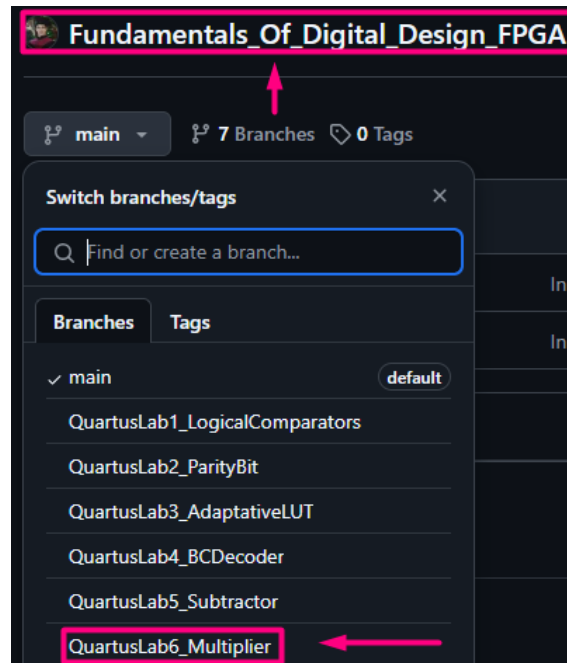


Figure: 12 GitHub Repository

3.- CHALLENGUES/ISSUES

Alonso Emmanuel López Macías

One of the main challenges was understanding the remaining logic of the multiplier, since in this practice we had to consider shifting a bit. At first, we thought about using or looking for a block that could handle that part before performing the calculation with logic gates, but we later realized that adding zeros could work for that section. It was a long circuit, and I believe it took us quite some time to find the solution.

Cesar Eduardo Inda Cenicerós

The most challenging aspects of this practice were understanding the multiplier logic from scratch and figuring out how to integrate the 8-bit adders we had previously created. We had to determine how to pass those buffer parameters to the partial sums with the input data. Additionally, when placing these blocks, we noticed that the "Cin" value i.e., the carry-in of the adders was carrying a "1". To discard this carry, we had to connect a GND to each "Cin" input of the adder blocks.

4.- CONCLUSIONS

Alonso Emmanuel López Macías

I personally learned a lot from this lab because it allowed me to understand in greater depth how a multiplier works. It also helped me see how closely it is related to other components we've studied so far in the course, and the importance of understanding each concept covered in class topics. From considering the adder for this circuit to learning the process of turning it into a block or symbol so it can be integrated into another project—as in this case—this helped us complete the logic. I also had more practice with the software and the board, realizing that it's not just about connecting wires, as we are often used to. At this design stage, it involves understanding programming and schematic design so that the logic is properly interpreted by Quartus, allowing correct compilation with the input values—such as connecting something to VCC or GND, as we encountered with the bit shifting.

Cesar Eduardo Inda Cenicerros

This lab broadened my perspective in the area of digital design fundamentals. I had previously had the opportunity to teach a PCB course to state-level teachers, but this part of digital design showed me a lower-level background—understanding the logic behind a component, or in this case, how logic gates come together to form integrated circuits or various known elements. Learning more about these logical, mathematical, and electronic processes has helped me realize the importance of understanding them deeply, as they are the foundation of all modern electronics. Personally, as an embedded systems security programmer, I know that cryptography implicitly relies on these processing mechanisms at such a fast level that they are sometimes imperceptible during debugging or in code at first glance. However, the process is essential and forms the core of all these systems. I personally consider it very important to learn these digital design fundamentals and to explore efficient methods and best practices when working with FPGA boards.